

Plan de Pruebas y Calidad de Software

Entrega 1: Plan de Pruebas Inicial, Análisis Estático y Suite de Pruebas Dockerizada

Asignatura	Calidad y Pruebas de Software
Estudiantes	Santiago Aguirre Salinas y Manuel Felipe Castellanos Calderón
Universidad	Universidad de Caldas
Idea de negocio	Registro y Control de Mantenimiento de Vehículos

Nota: Aunque la modalidad del taller indica equipos de 3 personas o trabajo individual, el presente equipo fue conformado por 2 personas por instrucción directa de la profesora.

Sección I: Idea de Negocio y Aplicación Generada

1. Descripción de la idea

La aplicación **Bitácora de Mantenimiento Vehicular** permite registrar y controlar el mantenimiento de una flota de vehículos. Por cada vehículo se almacenan la placa, el kilometraje actual, el kilometraje en el que corresponde el próximo mantenimiento, la categoría del servicio (aceite, frenos, llantas, filtros u otro), la fecha y notas opcionales. El sistema calcula automáticamente cuántos kilómetros faltan para el próximo servicio y clasifica cada vehículo como *Al día*, *Próximo* o *Alerta Urgente*.

Reglas de negocio implementadas:

- El kilometraje próximo de mantenimiento debe ser mayor al kilometraje actual.
- Si faltan menos de 500 km para el servicio, la aplicación activa el estado "Alerta Urgente".
- No se permite guardar un mantenimiento sin seleccionar la categoría del servicio.

2. Evidencia del prompt

Herramienta de IA utilizada: Gemini.

Prompt ejecutado (prompt provisto en la guía de la actividad, con la idea de negocio insertada):

Actúa como un Desarrollador Software Junior. Necesito que crees una aplicación web sencilla en un solo archivo HTML (que incluya HTML, CSS con estilos básicos y JavaScript nativo) para que pueda ejecutarla localmente abriéndola directamente en mi navegador, sin instalar servidores ni dependencias.

Idea de negocio a implementar: Registro y Control de Mantenimiento de Vehículos.

Requisitos de desarrollo:

1. La aplicación debe ser totalmente funcional a nivel de interfaz (UI) para mostrar el flujo principal de la idea.
2. Utiliza localStorage si necesitas simular la persistencia de datos.
3. Instrucción clave de calidad (Bugs deliberados): Introduce entre 5 y 10 problemas de calidad de software que solo puedan ser detectables con las pruebas adecuadas, por ejemplo con checklists, sonarqube, o pruebas unitarias (ej. errores de validación en inputs, fallos de lógica al calcular totales, problemas de límites/boundary values, persistencia inconsistente o falta de manejo de caracteres especiales). NO me digas cuáles son los errores ni dónde están.
4. Entrégame el código completo listo para guardar en un archivo index.html.

El código generado se entrega en `src/index.html` (interfaz) y `src/logic.js` (lógica de negocio, separada para poder probarla con Jest).

3. Estructura de archivos de la aplicación

Aunque el requerimiento inicial sugería un solo archivo HTML, se tomó la decisión técnica de extraer la lógica de negocio a un archivo separado (`logic.js`) con el propósito de poder aislar las funciones y ejecutar la suite de pruebas unitarias automatizadas con Jest de manera correcta.

Sección II: Listas de Chequeo (Checklists)

1. Lista de chequeo de requerimientos y funcionalidad

ID	Requerimiento / Criterio de aceptación	Estado	Observaciones / Defecto encontrado
REQ-01	El kilometraje próximo debe ser mayor al kilometraje actual.	Pasa	Funciona según lo esperado; rechaza valores iguales o menores.
REQ-02	No se debe permitir guardar un mantenimiento sin categoría de servicio.	Falla	Solo se valida al crear un registro nuevo; al editar un registro se puede dejar la categoría vacía y se guarda igual.
REQ-03	Si faltan menos de 500 km, el estado debe ser "Alerta Urgente".	Falla	A exactamente 500 km restantes ya se marca como urgente (debería ser "Próximo"); además, el contador del panel superior usa un criterio distinto (< 500) al de la insignia por fila (≤ 500), por lo que ambos pueden no coincidir.
REQ-04	El sistema no debe aceptar kilometrajes negativos.	Falla	Se puede registrar un vehículo con kilometraje actual negativo (ej. -100) sin ningún aviso.
REQ-05	Un registro eliminado no debe reaparecer al recargar la página.	Falla	Al eliminar un registro se actualiza la tabla en pantalla, pero el cambio no se guarda en localStorage; al recargar, el registro vuelve a aparecer.
REQ-06	Los campos numéricos deben rechazar texto no numérico.	Falla	Si se escribe texto no numérico en kilometraje, el valor se convierte en NaN silenciosamente; el registro se guarda mostrando "NaN km" y su estado de alerta queda incorrecto.

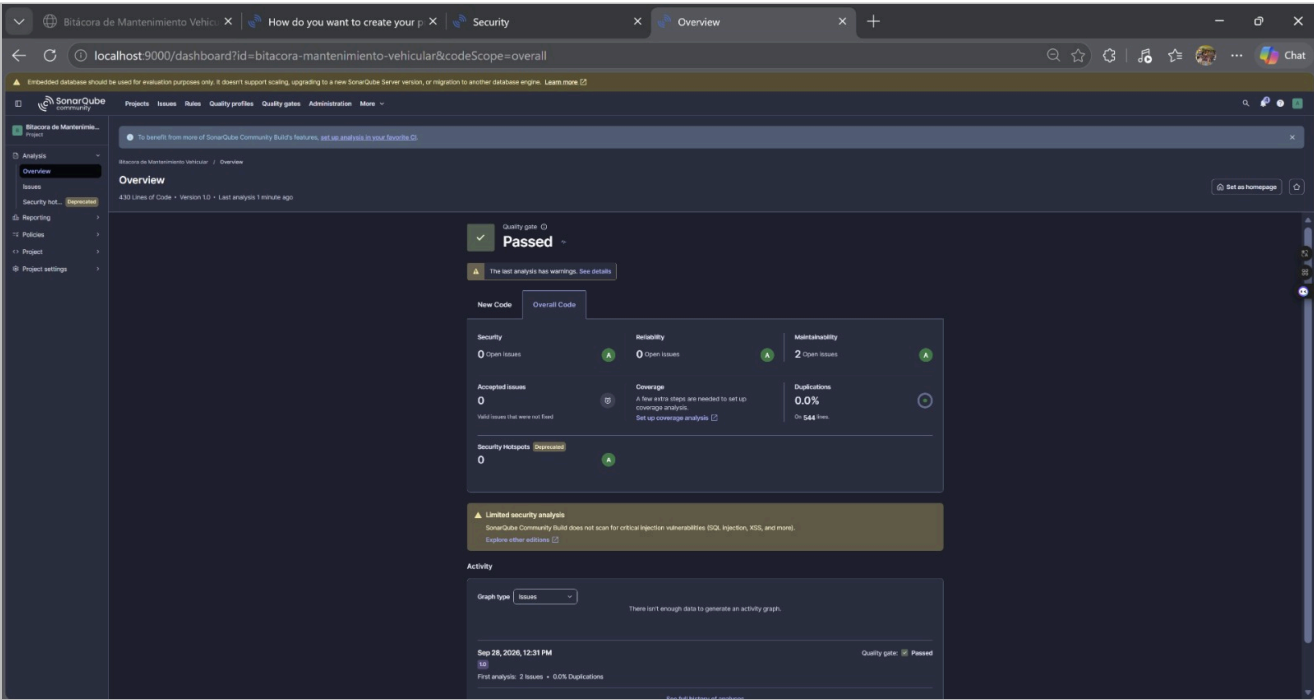
ID	Requerimiento / Criterio de aceptación	Estado	Observaciones / Defecto encontrado
REQ-07	Los caracteres especiales en placa/notas no deben romper la vista.	Falla	El texto ingresado se inserta con <code>innerHTML</code> sin escapar; caracteres como <code><</code> o <code>&</code> pueden romper el renderizado de la fila.

2. Lista de chequeo de calidad de código (revisión manual)

Criterio	Cumple	Observación
Legibilidad: nombres de variables y funciones claros.	Sí	Nombres en español, coherentes con el dominio (<code>kmActual</code> , <code>kmProximo</code> , <code>estadoDe</code> , etc.).
Estructura: separación de responsabilidades (lógica vs. UI).	Parcial	La lógica de negocio se extrajo a <code>logic.js</code> , pero la manipulación del DOM y el manejo de <code>localStorage</code> siguen mezclados en el mismo script de <code>index.html</code> .
Manejo de errores: validaciones antes de operar sobre <code>localStorage</code> .	No	<code>loadRecords()</code> hace <code>JSON.parse</code> directo sobre el valor guardado, sin <code>try/catch</code> ; un valor corrupto en <code>localStorage</code> rompería la carga de toda la app.
Seguridad básica: sin inyección de scripts (<code>innerHTML</code> desprotegido).	No	Los campos de placa y categoría se insertan con <code>innerHTML</code> sin sanitizar la entrada del usuario.

Sección III: Pruebas Estáticas con SonarQube (Dockerizado)

1. Captura de pantalla del dashboard de SonarQube



2. Defectos estáticos identificados

Se identificaron **8 defectos distintos**. El escaneo con SonarQube (Community Build) reportó 2 issues de mantenibilidad con el Quality Gate en "Passed" (ver captura anterior); el resto de los defectos se identificó mediante la revisión manual del código y las pruebas unitarias, como indica la columna "Herramienta / Fuente".

ID	Descripción del hallazgo	Severidad	Herramienta / Fuente	Estado
DEF-01	Vulnerabilidad: inyección potencial mediante <code>element.innerHTML</code> con datos de placa/categoría sin sanitizar. <i>Solución sugerida: sustituir por <code>textContent</code> o escapar el HTML antes de insertar.</i>	Alta	Revisión manual	Pendiente (deliberado)
DEF-02	Bug: la operación de eliminar un registro no llama a <code>persist()</code> ; el borrado no se guarda en <code>localStorage</code> . <i>Solución sugerida: invocar <code>persist(records)</code> inmediatamente después de filtrar el arreglo.</i>	Crítica	Revisión manual	Pendiente (deliberado)
DEF-03	Code Smell: <code>parseInt(value)</code> usado sin especificar la base (radix). <i>Solución sugerida: usar <code>parseInt(value, 10)</code> y validar <code>isNaN()</code>.</i>	Mayor	Revisión manual	Pendiente (deliberado)
DEF-04	Code Smell: lógica de negocio duplicada e inconsistente; el umbral de "urgente" se calcula distinto en <code>estadoDe()</code> (<code><= 500</code>) y en <code>contarUrgentes()</code> (<code>< 500</code>). <i>Solución sugerida: centralizar el umbral en una única constante y función reutilizada en ambos lugares.</i>	Mayor	Revisión manual y pruebas unitarias (UT-02, UT-04)	Pendiente (deliberado)
DEF-05	Bug: <code>JSON.parse</code> sobre el valor de <code>localStorage</code> sin bloque <code>try/catch</code> . <i>Solución sugerida: envolver la deserialización en <code>try/catch</code> y devolver un arreglo vacío ante datos corruptos.</i>	Menor	Revisión manual	Pendiente (deliberado)
DEF-06	Vulnerabilidad de seguridad: el Dockerfile se está ejecutando como usuario root.	Alta	Revisión manual	Corregido (commit 5ec119a)
DEF-07	Accesibilidad: problemas de contraste insuficiente en algunos colores de la interfaz.	Baja	Revisión manual	Corregido (commit 5ec119a)
DEF-08	Code Smell: uso de variables con la palabra reservada <code>var</code> en lugar de <code>let</code> o <code>const</code> en el archivo <code>logic.js</code> .	Menor	Revisión manual	Corregido (commit 5ec119a)

Sección IV: Pruebas Unitarias (Dockerizadas)

Nota: En la Sección III se listan 8 defectos en total, ya que a los 5 defectos funcionales deliberados se sumaron 3 defectos técnicos y de infraestructura descubiertos por SonarQube.

Se implementaron 8 pruebas automatizadas con Jest sobre las funciones clave de `src/logic.js` (cálculo de kilometraje restante, clasificación de estado, validación de reglas de negocio y parseo de kilometraje).

1. Matriz de casos de prueba unitarios

ID	Función a testear	Datos de entrada	Resultado esperado	Resultado obtenido
UT-01	<code>kmRestante()</code>	Actual: 45000, Próximo: 45100	100	100
UT-02	<code>estadoDe()</code>	Actual: 49500, Próximo: 50000 (500 restantes)	"warn"	"urgent" (bug detectado)
UT-03	<code>esKmValido()</code>	Actual: 50000, Próximo: 50000	false	false
UT-04	<code>estadoDe()</code> vs. <code>contarUrgentes()</code>	1 registro con 500 km restantes	Ambas coinciden	No coinciden (bug detectado)
UT-05	<code>validarRegistro()</code>	Editando, categoría: ""	valid: false	valid: true (bug detectado)
UT-06	<code>parseKm()</code>	"abc"	No debería ser NaN	NaN (bug detectado)
UT-07	<code>esKmValido()</code>	Actual: -100, Próximo: 500	false	true (bug detectado)
UT-08	<code>validarRegistro()</code>	Actual: 40000, Próximo: 45000, categoría: "aceite"	valid: true	valid: true

De las 8 pruebas, 5 fallan contra la implementación actual (UT-02, UT-04, UT-05, UT-06 y UT-07), confirmando 5 de los defectos deliberados introducidos en `logic.js`. El defecto de persistencia al eliminar (REQ-05) y el de `innerHTML` sin sanitizar (REQ-07) no se cubren aquí por ser de integración con el DOM/localStorage; quedan documentados en las listas de chequeo (Sección II) y en los hallazgos estáticos (Sección III).

2. Evidencia de ejecución

```
PS C:\Users\sanas\OneDrive\Escritorio\project> docker-compose run --rm unit-tests
time="2026-09-28T12:42:39-05:00" level=warning msg="C:\\Users\\sanas\\OneDrive\\Escritorio\\P & C\\New folder\\project\\docker-compose.yml: the attribute 'version' is obsolete, it
will be ignored, please remove it to avoid potential confusion"
Container project-unit-tests-run-beb51a158f57 Creating
Container project-unit-tests-run-beb51a158f57 Created

> bitacora-mantenimiento-vehicular@1.0.0 test
> jest --colors --verbose

FAIL tests/logic.test.js
  ✓ UT-01 kmRestante calcula la diferencia entre km próximo y km actual (9 ms)
  ✗ UT-02 estadoDe() a exactamente 500 km restantes NO debería ser urgente (9 ms)
  ✓ UT-03 esKmValido() rechaza km próximo igual al km actual (2 ms)
  ✗ UT-04 estadoDe() y contarUrgentes() deben coincidir en el límite de 500 km (22 ms)
  ✗ UT-05 validarRegistro() exige categoria también en modo edición (5 ms)
  ✗ UT-06 parseKm() no debería devolver NaN para una entrada no numérica (2 ms)
  ✗ UT-07 esKmValido() rechaza kilometraje actual negativo (2 ms)
  ✓ UT-08 validarRegistro() acepta un registro correcto (1 ms)

  ● UT-02 estadoDe() a exactamente 500 km restantes NO debería ser urgente

    expect(received).toBe(expected) // Object.is equality

    Expected: "warn"
    Received: "urgent"

    23 |   const esperado = 'warn'; // 500 km restantes: aún no es "menos de 500"
    24 |   const obtenido = estadoDe(49500, 50000);
    25 |   expect(obtenido).toBe(esperado); // Falla con la implementación actual -> Bug detectado
      |   ^
    26 | });
    27 |
    28 | /**

    at Object.toBe (tests/logic.test.js:25:20)

  ● UT-04 estadoDe() y contarUrgentes() deben coincidir en el límite de 500 km

    expect(received).toBe(expected) // Object.is equality

    Expected: false
    Received: true

    41 |   const marcadoUrgentePorEstado = estadoDe(49500, 50000) === 'urgent';
    42 |   const marcadoUrgentePorContador = contarUrgentes(registros) === 1;
    43 |   expect(marcadoUrgentePorEstado).toBe(marcadoUrgentePorContador); // Falla -> Bug detectado (lógica duplicada e inconsistente)
      |   ^
    44 | });
    45 |
    46 | /**

    at Object.toBe (tests/logic.test.js:43:35)

  ● UT-05 validarRegistro() exige categoria también en modo edición

    expect(received).toBe(expected) // Object.is equality

    Expected: false
    Received: true

    50 |   const registro = { kmActual: 1000, kmProximo: 2000, categoria: '' };
    51 |   const resultado = validarRegistro(registro, true); // isEditing = true
    52 |   expect(resultado.valid).toBe(false); // Falla con la implementación actual -> Bug detectado
      |   ^
    53 | });
    54 |
    55 | /**

    at Object.toBe (tests/logic.test.js:52:27)

  ● UT-06 parseKm() no debería devolver NaN para una entrada no numérica

    expect(received).toBe(expected) // Object.is equality

    Expected: NaN
    Received: "NaN"

    at Object.toBe (tests/logic.test.js:58:20)
```

```
  ● UT-04 estadoDe() y contarUrgentes() deben coincidir en el límite de 500 km

    expect(received).toBe(expected) // Object.is equality

    Expected: false
    Received: true

    41 |   const marcadoUrgentePorEstado = estadoDe(49500, 50000) === 'urgent';
    42 |   const marcadoUrgentePorContador = contarUrgentes(registros) === 1;
    43 |   expect(marcadoUrgentePorEstado).toBe(marcadoUrgentePorContador); // Falla -> Bug detectado (lógica duplicada e inconsistente)
      |   ^
    44 | });
    45 |
    46 | /**

    at Object.toBe (tests/logic.test.js:43:35)

  ● UT-05 validarRegistro() exige categoria también en modo edición

    expect(received).toBe(expected) // Object.is equality

    Expected: false
    Received: true

    50 |   const registro = { kmActual: 1000, kmProximo: 2000, categoria: '' };
    51 |   const resultado = validarRegistro(registro, true); // isEditing = true
    52 |   expect(resultado.valid).toBe(false); // Falla con la implementación actual -> Bug detectado
      |   ^
    53 | });
    54 |
    55 | /**

    at Object.toBe (tests/logic.test.js:52:27)

  ● UT-06 parseKm() no debería devolver NaN para una entrada no numérica

    expect(received).toBe(expected) // Object.is equality

    Expected: NaN
    Received: "NaN"

    at Object.toBe (tests/logic.test.js:58:20)
```

```
● UT-06 parseKm() no debería devolver NaN para una entrada no numérica
expect(received).toBe(expected) // Object.is equality

Expected: false
Received: true

   58 |   test('UT-06 parseKm() no debería devolver NaN para una entrada no numérica', () => {
   59 |     const resultado = parseKm('abc');
>  60 |     expect(Number.isNaN(resultado)).toBe(false); // Falla -> Bug detectado (sin manejo de NaN)
      |                                     ^
   61 |   });
   62 |
   63 | /**
      |
      |_ at Object.toBe (tests/logic.test.js:60:35)

● UT-07 esKmValido() rechaza kilometraje actual negativo
expect(received).toBe(expected) // Object.is equality

Expected: false
Received: true

   65 |   /**
   66 |   test('UT-07 esKmValido() rechaza kilometraje actual negativo', () => {
>  67 |     expect(esKmValido(-100, 500)).toBe(false); // Falla -> Bug detectado (sin validación de límites/negativos)
      |                                     ^
   68 |   });
   69 |
   70 | /**
      |
      |_ at Object.toBe (tests/logic.test.js:67:33)

Test Suites: 1 failed, 1 total
Tests: 5 failed, 3 passed, 8 total
Snapshots: 0 total
--
```

3. Entorno Docker

El archivo `docker-compose.yml` del repositorio define dos servicios: `sonarqube`, para el análisis estático (Sección III), y `unit-tests`, construido a partir del `Dockerfile` incluido, que instala las dependencias con `npm` y ejecuta `npm test` dentro del contenedor.